HacdsDeFi RPC Belgeleri

2026-11-11 Kontrat #3000 Geliştiriciler için

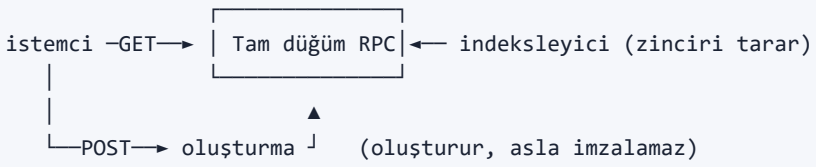
> Bu belge, Hacash ana ağındaki `HacdsDeFi3000.fitsh` kontratını açıklar (adres `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`, varlık serial 3000, ondalık 0).
>
> **Bu belge hiçbir #2000 dönemi arayüzü içermez.** Oracle, puanlama sertifikaları ve yönetici fonksiyonları yeni kontratta **artık yok** — yetkiler alındığı için değil, bu fonksiyonlar hiç yazılmadığı için.

---

0. Servislere genel bakış

hacds dört HTTP servisi içerir. İlk ikisi zinciri okur, diğer ikisi kolaylıktır.

Servis	Adres	Protokol	Amaç
Tam düğüm RPC	`node.hacds.com`	HTTP/JSON	Zincir verisi, sandbox salt okunur çağrı, doğrulama/yayın
API vekili	`api.hacds.com`	HTTP/JSON	Düğüme yönlendirir, CORS + HTTPS ekler
İndeksleyici	`indexer.hacds.com`	HTTP/JSON	Borçlu defteri (kim tasfiye edilebilir)
Oluşturma servisi	`build.hacds.com`	HTTP/JSON	İnsan girdisini imzasız işleme çevirir



API vekili neden gerekli: düğüm CORS başlığı döndürmez, tarayıcı doğrudan çağıramaz. `api.hacds.com` yalnızca yönlendirme ve başlık ekleme yapan bir Cloudflare Worker'dır; semantik değişmez. Kaynak portlar 80/443 olmadığı için Cloudflare kaynağa Tunnel üzerinden ulaşır.

Yanıt kuralı: her uç nokta üst düzeyde `ret` alanı olan JSON döndürür. `ret: 0` başarı, sıfırdan farklı değer başarısızlık (genellikle `err` ile).

> ⚠ **Başarısızlıklar da HTTP 200 döndürür.** Durum kodunu değil, gövdedeki `ret` alanını kontrol etmelisiniz.

Düğüm adresleri neden `IP:port` değil de alt alan adı: Cloudflare Workers

yalnızca 80 ve 443 portlarına ulaşabilir. `IP:8082` biçimindeki her adres, bağlantı denenmeden reddedilir (`error code: 1003`). Bu yüzden dışarıya her şey Tunnel destekli `*.hacds.com` alt alan adları üzerinden gider.

---

1. Tam düğüm API'si

1.1 Blok yüksekliği sorgusu

```
GET /query/block/height/latest
```

```
{"ret":0,"height":798812}
```

VM durumu (etkin mi, güncel yükseklik):

```
GET /api/vm/status
```

```
{"ret":0,"vm":"enabled","height":798812}
```

1.2 Ortalama ücret sorgusu

İşlem ücretini tahmin etmek için kullanılır.

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size (bayt)`. Düğüm kabulü `purity ≥ min_purity` gerektirir.

1.3 Bakiye sorgusu (HAC / elmas / varlık)

```
GET /query/balance?address={adres}&assets=true&asset_meta=true&diamonds=true
```

Üç bayrak da isteğe bağlıdır:

Bayrak	Etkisi
`assets=true`	Varlık listesini ekler (hacds buradadır)
`asset_meta=true`	Varlık adı / arzı gibi üst verileri ekler
`diamonds=true`	HACD elmas listesini ekler

```
{
  "ret": 0,
  "list": [{
    "address": "1P2tsn...",
    "hacash": "1234567:248",
    "diamond": 3,
    "assets": [{"serial": 3000, "amount": "100000000"}]
  }]
}
```

Tutarlar `<değer>:<birim>` biçimindedir: `5:249` 50 HAC demektir

(5 × 10^(249−248)), 5,0038 değil. Dönüşüm için §6.3.

1.4 Elmas detayı sorgusu

```
GET /query/diamond?name={6 karakterli elmas adı}
```

Elmasın numarasını, görsel genini, sahibini ve daha fazlasını döndürür.

Puanlama `number` ve `visual_gene` alanlarına ihtiyaç duyar; ikisi de buradadır.

1.5 Sandbox salt okunur çağrı (kontrat durumunu okumanın ana yolu)

```
GET /query/contract/sandboxcall?contract={kontrat}&function={fonksiyon}&params=
{parametreler}&caller={çağıran}
```

Salt okunur yürütme: durum yazılmaz, işlem gönderilmez, ücret ödenmez.

Gözlenen yanıt:

```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```

Alan	Anlamı
`ret`	0 = çağrı başarılı
`ret_val`	Fonksiyonun dönüş değeri
`use_gas` / `gas_use`	Tüketilen gaz, tahmin için

> ⚠ **Dağıtımdan önce burada `NotFindContract` döner.** Bu normaldir: kontrat 800.000 yüksekliğinde dağıtılır. Sonrasında çağrılabilir olur.

1.5.1 Bilmeniz gereken iki tuzak

Tuzak 1: `caller` geçmemek başarısız olur.

```
{"ret":1,"err":"[revert] address \"1111...\" balance 0:0 is insufficient, at least 559555:242"}
```

Varsayılan çağıranın bakiyesi sıfırdır ve gazı ödeyemez. `caller={kontrat adresi}` **geçin** — kontratın kendini çağırması bakiye gerektirmez.

Tuzak 2: IP başına sandbox eşzamanlılığı 2'dir. Fazlası **doğrudan reddedilir**, kuyruğa alınmaz:

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

İstemciler **sıralamalı + geri çekilmeli** çalışmalıdır. `Promise.all` ile bir düzine

istek atmayın.

1.5.2 Parametre biçimi

`params`, virgülle ayrılmış `değer:tür` çiftleridir. `:` karakteri `%3A` olarak

URL kodlanmalıdır.

Tür	Örnek
`u128`	`100000000:u128`
`address`	`1P2tsn...:address`
`bytes`	`WTYUIA:bytes`

1.5.3 Tam salt okunur fonksiyon listesi

AMM ve likidite

Fonksiyon	Parametreler	Döner	Not
`amm_reserves`	—	u128	Havuzdaki HAC (**zhu**)
`amm_reserve_ds`	—	u128	Havuzdaki hacds (**adet**)
`pool_price`	—	u128	Havuz fiyatı = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	Toplam LP payı
`lp_share_of`	`who:address`	u128	Bir adresin LP payı
`lp_pending_hac`	`who:address`	u128	Bekleyen LP ödülü (HAC, zhu)
`lp_pending_ds`	`who:address`	u128	Bekleyen LP ödülü (hacds, adet)
`total_staked`	`kind:u128`	u128	`0`=HAC, `1`=hacds

Kredi

Fonksiyon	Parametreler	Döner	Not
`loan_debt_of`	`who:address`	u128	Anapara (zhu), **tahakkuk etmemiş faiz hariç**

Fonksiyon	Parametreler	Döner	Not
`loan_debt_now`	`who:address`	u128	Faiz dâhil borç (zhu)
`loan_coll_of`	`who:address`	u128	Kilitli teminat (hacds adet)
`loan_health`	`who:address`	u128	**Sağlık, on binde bir**, aşağı bkz.
`loan_status`	`who:address`	u128	Teminat oranı ×100, aşağı bkz.
`total_debt`	—	u128	Küresel borç (zhu)
`providers_total`	—	u128	Toplam borç verilebilir fon
`utilization`	—	u128	Güncel kullanım (bps)
`borrow_coll_ratio`	—	u128	Borç çizgisi yüzdesi (`200`)
`liq_coll_ratio`	—	u128	Tasfiye çizgisi yüzdesi (`160`)
`borrow_paused`	—	u128	Sıfır değilse yeni borçlanma durdu
`ref_paused`	—	u128	Sıfır değilse fiyat sapması devre kesicisi etkin
`lendable_amm`	—	u128	AMM'in borç verilebilir kısmı (%30)
`net_claims`	—	u128	Ağ geneli toplam alacak
`reserve_hac`	—	u128	Batık borç rezervi
`deficit_hac`	—	u128	Açık

Sağlık nasıl doğru okunur

`loan_health` döner	Anlamı
`1000000`	Borç yok (kontratın döndürdüğü işaret değeri, çok sağlıklı sayın)
`> 10000`	Güvenli
`= 10000`	Tam tasfiye çizgisinde
`< 10000`	**Tasfiye edilebilir**
`< 9000`	Derin su; tam tasfiye mümkün

Önemli: bu değer `coll_to_hac_liq()` ile, yani **muhafazakâr fiyat** `min(spot, düzleştirilmiş referans)` ile hesaplanır ve `loan_liquidate` ile **aynı kaynaktan** gelir. Dolayısıyla "arayüz güvenli diyor, zincir tasfiye edilebilir diyor" durumu görülmez. Sağlığı `loan_status`'tan kendiniz hesaplamayın; bunu kullanın.

`loan_status` eski görünümdür (`coll_hac × 100 / debt` , spot fiyat), uyumluluk için korunmuştur:

Dönüş	Anlamı
`0`	Borç ve teminat yok
`999`	Teminat var, borç yok
diğer	Teminat oranı ×100

HACD havuzu

Fonksiyon	Parametreler	Döner	Not
`hacd_score_total`	—	u128	Havuzdaki toplam HACD puanı (× `SCORE_SCALE`)
`hacd_count`	—	u128	Havuzdaki HACD adedi
`hacd_avg_score`	—	u128	Ortalama puan (× `SCORE_SCALE`)
`hacd_coverage`	—	u128	Kapsam (bps, 10000 = %100)
`quote_redeem`	`sc:u128`	tuple	Geri alma teklifi, aşağı bkz.
`quote_score`	`name:bytes,vg:bytes,num:u128`	u128	Ad + gen + numaradan puan

`quote_redeem` **dönüşü**: `tuple(charge, fee, premium_bps, ok)`

Öğe	Anlamı
`charge`	Toplam kesinti (anapara + ücret)
`fee`	Ücret kısmı
`premium_bps`	Kiraz toplama primi (bps)
`ok`	`1` = geri alınabilir; `0` = gerçek işlem `61` döndürür

Fiyat ve faktörler

Fonksiyon	Parametreler	Döner	Not
`ref_price_of`	—	u128	Düzleştirilmiş referans fiyat
`ref_price_zhu_per_ds`	—	u128	Referans fiyat (hacds başına zhu)
`ref_height`	—	u128	Referansın son güncellendiği blok
`hcut`	—	u128	HAC sosyalizasyon faktörü (1e12 ölçek, yalnızca azalır)
`lcut`	—	u128	LP ödülü sosyalizasyon faktörü

Hesaplar ve hazine

Fonksiyon	Parametreler	Döner	Not
`balance_hac`	`who:address`	u128	Etkin HAC bakiyesi (zhu, hcut ile çarpılmış)

Fonksiyon	Parametreler	Döner	Not
`balance_hac_nominal`	`who:address`	u128	Nominal pay (zhu)
`balance_ds`	`who:address`	u128	Bekleyen hacds kotası (due)
`bad_of`	`who:address`	u128	Adrese atfedilen batık borç
`treasury_pending`	`kind:u128`	u128	Hazinenin süpürülmeyi bekleyen tutarı
`treasury_ds`	—	u128	Birikmiş hazine hacds
`tot_hac`	—	u128	Toplam HAC (etkin)
`tot_hac_nominal`	—	u128	Toplam HAC (nominal)
`lpr_hac_effective`	—	u128	Etkin LP ödül havuzu

Operasyon

Fonksiyon	Parametreler	Döner	Not
`self_check`	—	u128	İç tutarlılık kontrolü, `0` = geçti
`rent_key_group_count`	—	u128	Anahtar grubu sayısı (sabit `7`)
`rent_key_len_at`	`idx:u128`	u128	idx grubunun anahtar uzunluğu
`rent_keys_group`	`klen:u128`	bytes	O uzunluktaki tüm anahtarlar

1.5.4 Gözlenen örnek

```
GET {node}/query/contract/sandboxcall?contract=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
    &function=loan_health&params=1P2tsn...%3Aaddress&caller=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":1000000}
```

`ret_val: 1000000` = bu adresin borcu yok.

1.6 Blok özeti sorgusu

İndeksleyici, borçluları bulmak için blokları bununla gezer.

```
GET /query/block/intro?height={yükseklik}&tx_hash_list=true
```

1.7 İşlem sorgusu

```
GET /query/transaction?hash={işlem özeti}
```

1.8 İşlem doğrulama (okunabilir açıklamayla)

```
POST /util/transaction/check
Content-Type: application/json

{"tx": "040000... (hex)"}
```

Başarıda ayrıştırılmış sonucu ve okunabilir eylem açıklamasını döndürür:

```
{"ret":0,"describe":"[CallContract] T1A8... -> swap_hac_to_ds(...)"}
```

Bu uç nokta imzalamaz ve yayınlamaz; yalnızca doğrular. Cüzdanlar yayından önce son kapı olarak kullanır.

1.9 İşlem yayını

```
POST /submit/transaction
Content-Type: application/json

{"tx": "040000... (hex)"}
```

```
{"ret":0}
```

> ⚠ `ret: 0` **yalnızca belleğe (mempool) ulaştığını** gösterir, onaylandığını değil. Hedef blok süresi 300 saniyedir; `/query/transaction`'ı bu tempoda yoklayın.

1.10 Kontrat depolama maliyeti parametreleri

```
GET /query/contract/storage_cost?contract={kontrat}
```

Yenileme maliyetini tahmin etmek için.

---

2. İndeksleyici API'si

İndeksleyici zinciri tarar ve "kimin borcu var, kim tasfiye edilebilir" defterini tutar. **Cüzdanlar zinciri kendileri taramaz.**

2.1 Sağlık kontrolü

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 798812, "watched": 0}
```

2.2 Defter anlık görüntüsü (çoğu istemcinin ihtiyaç duyduğu tek uç nokta)

```
GET /snapshot
```

İzlenen her adresin sağlık anlık görüntüsünü döndürür:

```
{
  "head": 798812,
  "updated": 1762900000,
  "items": [
    {"addr": "1P2tsn...", "debt": "100000000", "coll": "300000000",
     "health": 16000, "ratio": 160, "liq": true}
  ]
}
```

Alan	Anlamı
`debt`	`loan_debt_now` (faiz dâhil)
`coll`	Teminat hacds adedi
`health`	On binde bir, `loan_health` ile aynı ölçek
`ratio`	Teminat oranı yüzdesi
`liq`	`true` = şu anda tasfiye edilebilir

2.3 Yalnızca anlık görüntü (senkronizasyon yok)

```
GET /snapshot?cached=1
```

2.4 Zorla senkronizasyon

```
POST /sync
```

En son bloğa kadar hemen tarar. Operasyon içindir; normalde gerekmez.

2.5 İzleme listesi bakımı

```
POST /watch/add?addr={adres}
POST /watch/remove?addr={adres}
```

İndeksleyici borçluları otomatik bulur; bu uç noktalar boşlukları elle kapatır.

2.6 Arz kapsamı izleme

```
GET /backing
```

"Kontratın kaydettiği toplam due" ile "zincirde gerçekten basılmış hacds"i

karşılaştırır, anormallik tespiti için.

2.7 Kira partileri

```
GET /rent_batches
```

Uzunluğa göre gruplanmış ve doğrudan `rent_keys`'e verilebilecek anahtar partilerini döndürür. Keeper bunu kullanır.

2.8 Dağıtım

```
python3 hacds_indexer.py --node http://127.0.0.1:8082 \
    --contract T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH --port 8124
```

---

3. Oluşturma servisi

Bu servis "insan tarafından okunabilir parametreleri" "imzasız işleme" çevirir.

Hiçbir özel anahtar tutmaz; oluşturduğu işlem istemci tarafından imzalanmalıdır.

Kaynak: `hacds_exchange/indexer/hacds_build_server.py`

3.0 Neden var

Hacash işlemi oluşturmak hassas ücret / gaz / eylem kodlaması ve elmas adı listelerinin wire biçimi gerektirir. Her istemcinin bunu yeniden uygulaması, üç istemcinin farklı davranacağını garanti eder. Oluşturmayı merkezileştirmek, istemcinin yalnızca imzalaması demektir — **imzalanan şey değişir, imzalayan değişmez.**

3.1 Sağlık kontrolü

```
GET /health
```

```
{"ok": true, "service": "hacds-build", "version": "2026-11-11.2",
 "txbuild_ok": true, "contract": "T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH"}
```

`version` önemlidir: Python sıcak yeniden yükleme yapmaz, değişen bir betik süreç yeniden başlatılmasını gerektirir. Davranış ile `version` uyuşmuyorsa önce sürecin yeniden başlatıldığını doğrulayın.

3.2 Kontrat çağrısı oluşturma

```
POST /api/build
Content-Type: application/json
```

```
{
  "mode": "call",
  "main": "1P2tsn...",
  "fn": "swap_hac_to_ds",
  "params": "100000000:u128,0:u128,0:u128",
  "fee": "0:248",
  "gasMax": 99
}
```

Alan	Zorunlu	Not
`mode`	Hayır	`call` (varsayılan) veya `transfer`
`main`	✅	İşlemi başlatan adres (ücreti öder)
`fn`	✅	Kontrat fonksiyon adı, beyaz liste §3.4
`params`	Hayır	`değer:tür` virgülle ayrılmış, varsayılan `-`
`fee`	Hayır	`0:248` biçiminde
`gasMax`	Hayır	`1–99` arası tam sayı, varsayılan `99`

Yanıt:

```
{"ok": true, "unsigned_tx": "040000...", "fn": "swap_hac_to_ds",
 "fee": "0:248", "gas_max": 99}
```

3.3 Transfer oluşturma

```
POST /api/build
{
  "mode": "transfer",
  "main": "1P2tsn...",
  "sub": "hacd-from-to",
  "fee": "0:248"
}
```

`sub`, `hacds_txbuild transfer` alt komutudur; ek argümanlar alt komuta göre verilir. `from` **kontrat adresine kilitlidir** — bu servisin "rastgele bir adresten transfer" işlemi oluşturmasını engelleyen bir güvenlik kısıtıdır.

3.4 Fonksiyon beyaz listesi

```
swap_hac_to_ds · swap_ds_to_hac
add_liquidity · remove_liquidity · lp_harvest
loan_add_collateral · loan_borrow · loan_repay · loan_liquidate
rent_keys · topup_keys
```

Beyaz liste dışındaki fonksiyonlar reddedilir. **Bu bir beyaz listedir, kara liste değil** — yeni fonksiyonlar açıkça eklenmelidir.

3.5 Sınırlar

Sınır	Değer	Neden
IP başına dakikada istek	30	Kötüye kullanımı önleme
İstek gövdesi	8 KB	Parametre 1024 bayt, en fazla 16 öğe
Elmas adı listesi	En fazla 200	`MAX_DEPOSIT_COUNT` ile eşleşir
`gasMax`	1–99	Kontrat ve motor için geçerli aralık

3.6 Hata kodları

`error`	Anlamı
`bad_json`	Gövde geçerli JSON değil
`bad_length`	Gövde boş veya 8 KB üzeri
`bad_main`	`main` geçerli adres değil
`bad_fee`	`fee` `değer:mantis` biçiminde değil
`bad_gasMax`	`gasMax` 1–99 arası tam sayı değil
`fn_not_allowed`	Fonksiyon beyaz listede değil
`bad_params`	Parametre dizisi bozuk
`rate_limited`	Dakikada 30 isteği aştı
`build_timeout`	Motor zaman aşımı (30 saniye)
`engine_missing`	Sunucuda `hacds_txbuild` bulunamadı

---

4. Borsa listeleme entegrasyonu

4.0 Tek bir şeyi unutmayın: amount düz adettir

hacds **ondalık 0**'dır. Zincirden dönen `amount` adedin kendisidir, **hiçbir dönüşüm yoktur**.

```
{"serial": 3000, "amount": "100000000"}   →  100.000.000 hacds
```

Önceki sürüm ondalık 8'di ve 1e8'e bölmek gerekiyordu. **Bu sürümde bölmeyin.**

4.1 Varlık kimliği (listeleme kaydı için)

Alan	Değer
serial	`3000`
ticket	`HACDS`
name	`hacds`
ondalık	`0`
arz	`5963298700000000`
issuer	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`

⚠ Ayrıca ticket'ı HACDS olan serial `1028` varlığı var (adı "HACD Fragments", başkasının). **Ticket'a göre değil, serial'e göre eşleştirin.**

4.2 Yatırma tespiti

Blokları tarayıp `serial == 3000` ve `to == yatırma adresiniz` olan `TransferAssetFromTo` eylemlerini bulun.

```
GET /query/block/intro?height={H}&tx_hash_list=true
  → tx_hash_list üzerinde gezinin, her biri için GET /query/transaction?hash={h}
  → varlık transfer eylemlerine bakın
```

Önerilen onay sayısı: 3 blok (yaklaşık 15 dakika). Hedef blok süresi 300 saniyedir ama dalgalanır.

4.3 Çekim oluşturma

Çekim standart bir varlık transferidir:

```
hacds_txbuild transfer {main} {ts} asset-from-to {from} {to} 3000 {amount}
```

> ⚠ **`hacds` takma adı yerine sayısal serial `3000` kullanın.**
> `hacds_txbuild` içindeki `hacds` takma adı hâlâ eski serial `2000` değerine işaret eder ve #3000 sözleşmesindeki `HACDS_SERIAL = 3000` ile uyuşmaz; zincir bunu `PermitAsset` içinde `return 1` ile reddeder.
> `/api/build` servisi `hacds` değerini otomatik olarak `3000` yapar, ancak **doğrudan komut satırı çağrısı** bu korumadan yararlanmaz.

Ya da oluşturma servisi üzerinden (`mode=transfer`, `sub=asset-from-to`).

4.4 Yayın ve onay

```
POST {node}/submit/transaction
{"tx": "...hex..."}
```

`ret:0` yalnızca belleğe ulaştığı anlamına gelir. Blok yüksekliğine göre `/query/transaction?hash=` ile yoklayın; `block_height` göründüğünde onaylanmıştır.

4.5 Ücretler ve gaz

Öğe	Not
fee biçimi	`değer:mantis`, ör. `0:248` (en az), `2:247` (0,02 HAC)
fee formülü	`fee = purity × billing_size`
`gasMax`	`1–99`, varsayılan 99. **Kontrat çağrılarında 99'un altına inmeyin**

4.6 Bilmeniz gereken üç sert sınır

① **İşlem en fazla 16.384 bayttır.** Bu derleme zamanı sabitidir. 200 elmaslık yatırma bu sınıra yaklaşır.

② **IP başına sandbox eşzamanlılığı = 2.** Fazlası kuyruğa alınmaz, reddedilir. Borsa arka uçları sıralamalı çalışmalıdır.

③ **Üç gaz kovası**: compute 18.009 / resource 6.100 / storage 111.911. Aşımda `OutOfGas(31): <kova> N > M` raporlanır. **`gasMax`'ı artırmak işe yaramaz** — o toplam bütçedir; kova başına sınır sert bir kısıttır.

> #3000'ün "işlem başına 1 elmas"tan "işlem başına 200 elmas"a çıkabilmesinin nedeni tam olarak puanlamanın RV64IM çekirdeğine taşınmasıdır: 200 elmas `compute`tan yalnızca 3.587 harcar, tek kova. Eski FitSH sürümü tek elmas için 3.270 `resource` harcıyordu, yani iki elmas 6.100'ü aşıyordu.

4.7 Piyasa yapıcılığı / fiyat teklifi için gereken veriler

```
GET {node}/query/contract/sandboxcall?contract={kontrat}&function=amm_reserves&caller={kontrat}
GET {node}/query/contract/sandboxcall?contract={kontrat}&function=amm_reserve_ds&caller={kontrat}
```

Spot fiyat (hacds başına zhu) = `amm_reserves × 1e8 / amm_reserve_ds`.

Toplu çekmek daha kolaydır:

Fonksiyon	Bir seferde döner
`ref_price_zhu_per_ds`	Düzleştirilmiş referans fiyat
`pool_price`	Havuz fiyatı (×1e8)
`total_debt` / `utilization`	Kredi kullanımı
`hacd_count` / `hacd_avg_score`	Havuz büyüklüğü

4.8 Mutabakat

Zincir üstü özdeşlik:

```
kullanıcı due toplamı + AMM havuzu + hazine birikimi + LP ödül havuzu
    == zincirde basılmış toplam hacds
```

Ayrıca:

```
etkin HAC bakiyeleri toplamı × hcut == nominal paylar toplamı
```

İndeksleyicinin `GET /backing` uç noktası bunu her senkronizasyonda kontrol eder.

Uyuşmazlık bir hata demektir.

4.9 Listeleme kontrol listesi

- [ ] serial `3000` ile kaydedin, ondalık `0` (**8 değil**)
- [ ] Yatırmalar yalnızca serial 3000'i tanısın; aynı ticket'lı #1028'i yok sayın
- [ ] Çekimler `asset-from-to` kullansın, `serial` olarak `hacds` veya `3000`
- [ ] Onay sayısı ≥ 3
- [ ] Arka uç sandbox çağrılarını sıralamalı yapsın (eşzamanlılık sınırı 2)
- [ ] Büyük çekimlerde işlemin 16.384 bayt sınırına çarpıp çarpmadığı kontrol edilsin
- [ ] Mutabakat `/backing` ve zincir üstü özdeşlikle yapılsın

---

5. Hata yönetimi

5.1 Kontrat dönüş kodları

Sandbox çağrısından dönen sıfırdan farklı `ret_val`, kontrat içindeki bir `return N`'dir.

Genel

Kod	Anlamı
`2`	Bozuk parametre (ör. HACD ad listesi wire biçimi)
`48`	Adet aralık dışı (0 elmas / 200 üzeri / geri alma tam 1 değil)
`5`	Puan 0
`53`	Toplam arzı aşacak
`54`	Kapsam yetersiz (%100 basımda hiç tetiklenmez)
`59`	Süresi dolmuş (`deadline`)
`61`	HACD havuz kaydı yok / puan yetersiz (geri alma)

AMM

Kod	Anlamı
`2`	Bir taraf sıfır
`23`	Havuz dolu ama pay sıfıra yuvarlanıyor (ölçek sıfırlama koruması)
`24`	Hesaplanan pay sıfır (oran çok küçük)
`25`	LP payı yetersiz
`26`	İlk yatırım `1e18` taraf sınırını aşıyor
`56`	Kayma koruması tetiklendi (`min_out` / `min_share` tutmadı)

Kredi

Kod	Anlamı
`40`	Borç limitini aşıyor (`(debt+hac) × %200 > coll_hac`)
`41`	Borç sıfır (geri ödeme)
`50`	Küresel kullanım %50 üzeri
`51`	Borçlanma durdu (kullanım dolu veya fiyat devre kesicisi)
`52`	Tek borçlu sınırı aşıldı (%20)

Tasfiye

Kod	Anlamı
`42`	O adresin borcu yok
`43`	Henüz tasfiye edilebilir değil
`44`	Asgari tasfiye tutarının altında (1 HAC)
`45`	Teminat çıkışı `min_coll_out` altında
`46`	Derin su tasfiyesi tam geri ödeme gerektirir (`repay < deep_pay`)
`47`	Referans fiyat başlatılmamış; tasfiye reddedildi

Yönetici fonksiyonları yok

`set_fee` / `set_loan_rate` / `set_liquidate_bonus` / `set_paused` / `set_oracle`

#3000'de **uygulanmadı**. Çağrılmaları şunu döndürür:

```
CallNotExist(51)
```

Bu yetki hatası değil — **fonksiyon yok.** Bu çağrıları yazmayı bırakın.

5.2 Düğüm / sandbox hataları

Dizgiyle eşleştirin; yaygın sınıflar:

Ham mesaj	Anlamı	İşlem
`NotFindContract`	O adreste kontrat kodu yok	Dağıtımdan önce normal; sonra adresi kontrol edin
`invalid address argument` / `InvalidChecksum`	Adres biçimi veya sağlama hatası	Tam adresi yeniden alın
`balance ... is insufficient`	Çağıran gazı ödeyemiyor	`caller={kontrat adresi}` geçin
`sandbox_per_ip_concurrency_exceeded`	Eşzamanlılık aşıldı	Sırala + geri çekil
`resource gas limit exceeded`	Resource kovası aşıldı	İşlemi bölün; **gas_max artırmak işe yaramaz**
`compute gas limit exceeded`	Compute kovası aşıldı	İşlemi bölün
`storage gas limit exceeded`	Storage kovası aşıldı	Daha az yeni anahtar veya daha kısa kira
`OutOfGas(31)`	Bir kova tükendi	Parantezdeki kova adına bakın, bölün
`Arithmetic(90)`	Aritmetik alt/üst taşma	Girdileri kontrol edin; bu bir kontrat hatasıdır, bildirin
`CallNotExist`	Kontratta bu fonksiyon yok	Adı kontrol edin (yönetici fonksiyonları kaldırıldı)
`state changed`	Zincir üstü durum değişti	Yeniden deneyin
`transaction parse failed`	İşlem gövdesi ayrıştırılamadı	Eksik hex / bayt
`StorageNotActive`	Depolama anahtarının süresi doldu	`rent_keys` ile yenileyin
`ECONNREFUSED` / `timeout`	Servise ulaşılamıyor	Çalışıp çalışmadığını kontrol edin

Pratik kural: bunların çoğu **HTTP 200 +** `{"ret":1,"err":"..."}` biçimindedir.

Yalnızca durum kodunu kontrol etmek bunları kaçırır.

---

6. Hızlı başvuru

6.1 Uç nokta özeti

```
# Tam düğüm
GET  /query/block/height/latest
```

```
GET  /query/fee/average
GET  /query/balance?address={A}&assets=true&diamonds=true
GET  /query/diamond?name={N}
GET  /query/contract/sandboxcall?contract={C}&function={F}&params={P}&caller={C}
GET  /query/block/intro?height={H}&tx_hash_list=true
GET  /query/transaction?hash={H}
POST /util/transaction/check        {"tx": "hex"}
POST /submit/transaction            {"tx": "hex"}

# İndeksleyici
GET  /health
GET  /snapshot
POST /sync
GET  /backing
GET  /rent_batches

# Oluşturma servisi
GET  /health
POST /api/build
```

6.2 Sabitler

Sabit	Değer
Kontrat adresi	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`
Kontrat nonce	`2`
Hazine	`1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`
hacds serial	`3000`
hacds ondalık	`0`
hacds toplam arz	`5963298700000000`
`SCORE_SCALE`	`100000000` (1e8)
`MAX_DEPOSIT_COUNT`	`200`
`FEE_BPS`	`50` (%0,5)
`BORROW_RATIO_BPS`	`20000` (%200)
`LIQ_COLL_RATIO_BPS`	`16000` (%160)
`CLOSE_FACTOR_BPS`	`5000` (%50)
`DEEP_TRIGGER_BPS`	`10500` (%105)
`LIQ_BONUS_BPS`	`500` (%5)
`U_MAX_BPS`	`5000`
`LENDABLE_AMM_BPS`	`3000`
VM kovaları	compute 18.009 / resource 6.100 / storage 111.911

Sabit	Değer
İşlem sınırı	16.384 bayt
gasMax aralığı	1–99

6.3 Birim dönüşümü

HAC (zincirde zhu):

```
HAC = zhu / 1e8
"5:249" = 5 × 10^(249−248) = 50 HAC
"1:248" = 1 HAC
```

hacds (ondalık 0):

```
hacds adedi == zincirdeki tam sayı, dönüşüm yok
```

⚠ **hacds'i 1e8'e bölmeyin.** Bu önceki sürümün (ondalık 8) yöntemiydi.

6.4 Minimal çalışan durum okuyucu

```
const NODE = 'https://api.hacds.com';
const CONTRACT = 'T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH';

async function call(fn, params = '') {
  const q = new URLSearchParams({
    contract: CONTRACT,
    function: fn,
    params,
    caller: CONTRACT,       // geçilmezse yetersiz bakiye hatası verir
  });
  const r = await fetch(`${NODE}/query/contract/sandboxcall?${q}`);
  const j = await r.json();
  if (j.ret !== 0) throw new Error(j.err || 'call failed');
  return j.ret_val;
}

// Havuzun iki tarafı
const hacZhu = await call('amm_reserves');
const ds     = await call('amm_reserve_ds');

// Spot fiyat (hacds başına zhu)
const spot = Number(hacZhu) * 1e8 / Number(ds);
```

Eşzamanlılık notu: her IP'nin yalnızca 2 eşzamanlılık yuvası vardır. Toplu okumaları kuyruğa alın; `Promise.all` kullanmayın.

Bu belge, `HacdsDeFi3000.fitsh` kontratının 2026-11-11 tarihli kesinleşmiş sürümüne karşılık gelir.